

11 Installation, operation and release

11.1 Local operation and reviewer walkthrough

Prerequisites are the complete extracted source tree and a supported Node/npm environment. The source specifies Node ≥ 22.13 ; CI is configured for Node 22. The current review used Node 24.19.0. Run commands from the application root, where `package.json` is located. Account creation is not required for local D1 emulation.

```
npm ci
npm run db:migrate:local
npm run check
npm run dev
```

Open the local URL printed by the development server, normally `http://localhost:5173`. Select a fictional example; inspect its profile and applicable criteria; change a rating and evidence note; save and reopen; inspect gaps and download the text report. For a controlled validation check, select Documented with a short note and confirm rejection. Check the notice explaining local storage, and use Reset my demo only after exporting any work to retain.

Docker Compose procedure

The supplied Compose file starts the development server and keeps dependencies and local D1 state in named volumes. It does not apply the migration automatically. The following sequence adds the necessary database-initialisation step (Docker, n.d.):

```
docker compose build
docker compose run --rm app npm run db:migrate:local
docker compose up
```

Open `http://localhost:5173`. Use `docker compose down` to stop the environment while preserving named volumes. Removing volumes also removes local database state. Docker execution was not available in this review, so this procedure is a corrected rehearsal instruction, not a completed test result.

Common operational failures

Symptom	Specific response
Projects fail to load or API returns 503.	Check that the local or remote migration was applied to the intended D1 database and that the DB binding matches configuration.
A local save reports a stale revision.	Reload the latest project before reapplying the edit; do not overwrite newer content blindly.
Documented evidence is rejected.	Supply a meaningful fictional reference of at least ten characters after trimming, within the evidence length limit.
Port 5173 is already in use.	Select a free local port using the development command's port option; open the URL the server prints.

11.2 GitHub, Cloudflare and final source freeze

GitHub stores source and runs workflows; uploading a ZIP alone does not provision the Worker or its database. The inspected export includes Cloudflare deployment material, but no successful public deployment is evidenced. The sequence below is the remaining release procedure after the local write-isolation checks (Cloudflare, n.d.; GitHub, n.d.-a).

Step	Required action and exit evidence
1. Prepare source	Extract the archive and upload the project files with their directory structure, including workflow/configuration files. Keep installed dependencies, local database state and tokens out of Git.
2. Provision Cloudflare	Create or select the account and D1 database; obtain a token with the Worker deployment and D1 permissions needed by this project, restricted to the intended account.
3. Set deployment values	Supply CLOUDFLARE_ACCOUNT_ID, CLOUDFLARE_API_TOKEN and CLOUDFLARE_D1_DATABASE_ID using the repository deployment settings expected by the supplied workflow.
4. Check configuration	Confirm the Worker name, database binding and database identifier. The supplied configure-deployment script fills configuration from environment values; do not commit the API token.
5. Deploy and verify	Run the supplied deploy-cloudflare workflow; apply the remote migration and publish the Worker. Open the resulting HTTPS URL and execute the core workflow plus isolation checks.
6. Freeze the release	Record the actual public repository URL, application URL, successful workflow and immutable source commit. Tag the accepted release and create the implementation ZIP from that same commit.

The account ID identifies the account, the D1 database ID identifies the database, and the API token authorises deployment operations. They are not a reviewer's application login. Only the token is an authentication secret; this workflow may store all three values as GitHub secrets for consistent configuration. The academic demonstration is intended to open without an application account.

Source equivalence and submission integrity

Compare the relative file names and file-content hashes in the final implementation ZIP with an archive of the recorded Git commit, ignoring only the archive's enclosing directory. Do not compare only the ZIP byte hashes: regenerated source archives may use different compression while containing identical files (GitHub, n.d.-b). Include required source, assets, migrations, configuration and documentation; exclude secrets and generated local state.

The university requires GitHub and the submitted implementation source to remain identical and unchanged after final submission (IU, n.d., pp. 4–5). Complete repairs, deployment verification and packaging before the freeze; preserve the assessed release thereafter.